

UNIVERSIDADE FEDERAL DE OURO PRETO
INSTITUTO DE CIÊNCIAS EXATAS E BIOLÓGICAS
DEPARTAMENTO DE COMPUTAÇÃO

NIEGE MARRYANY DOS REIS DA SILVA
Orientador: Prof. Dr. Rodrigo Cesar Pedrosa Silva

**COMPROMISSO ENTRE PRIVACIDADE E UTILIDADE EM DEFESAS
CONTRA ATAQUES DE INVERSÃO DE GRADIENTES
REVISÃO DE LITERATURA**

Ouro Preto, MG
2026

UNIVERSIDADE FEDERAL DE OURO PRETO
INSTITUTO DE CIÊNCIAS EXATAS E BIOLÓGICAS
DEPARTAMENTO DE COMPUTAÇÃO

NIEGE MARRYANY DOS REIS DA SILVA

**COMPROMISSO ENTRE PRIVACIDADE E UTILIDADE EM DEFESAS CONTRA
ATAQUES DE INVERSÃO DE GRADIENTES
REVISÃO DE LITERATURA**

Revisão de literatura da Monografia I apresentada ao Curso de Ciência da Computação da Universidade Federal de Ouro Preto como parte dos requisitos necessários para a obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Prof. Dr. Rodrigo Cesar Pedrosa Silva

Ouro Preto, MG
2026

Lista de Ilustrações

Figura 2.1 – Exemplo guia: modelo linear com quatro entradas e três classes, cuja classe correta é gato.	4
Figura 2.2 – Fluxo de uma rodada de aprendizado federado: o servidor distribui o modelo global (azul) e os clientes devolvem apenas atualizações ou gradientes (vermelho), mantendo os dados locais.	8
Figura 2.3 – Princípio dos ataques de inversão de gradientes baseados em otimização. . .	9
Figura 2.4 – Efeito das defesas sobre o gradiente do exemplo guia e rótulo inferido pela regra do iDLG. A primeira linha é o gradiente original; células em vermelho contêm ruído, células em azul são valores mantidos e zeros em cinza são valores descartados.	18

Lista de Tabelas

Tabela 2.1 – Conjuntos de dados de imagens utilizados como referência.	7
--	---

Lista de Algoritmos

- 2.1 Federated Averaging (FedAvg) 8
- 2.2 Deep Leakage from Gradients (DLG) 10
- 2.3 DP-SGD 14

Lista de Abreviaturas e Siglas

ABNT	Associação Brasileira de Normas Técnicas
CNN	<i>Convolutional Neural Network</i> (Rede Neural Convolucional)
DECOM	Departamento de Computação
DGC	<i>Deep Gradient Compression</i>
DLG	<i>Deep Leakage from Gradients</i>
DP	<i>Differential Privacy</i> (Privacidade Diferencial)
DP-SGD	<i>Differentially Private Stochastic Gradient Descent</i>
FedAvg	<i>Federated Averaging</i>
FedSGD	<i>Federated Stochastic Gradient Descent</i>
iDLG	<i>Improved Deep Leakage from Gradients</i>
L-BFGS	<i>Limited-memory Broyden–Fletcher–Goldfarb–Shanno</i>
LDP	<i>Local Differential Privacy</i> (Privacidade Diferencial Local)
MLP	<i>Multilayer Perceptron</i> (Perceptron Multicamadas)
MSE	<i>Mean Squared Error</i> (Erro Quadrático Médio)
QSGD	<i>Quantized Stochastic Gradient Descent</i>
RDP	<i>Rényi Differential Privacy</i> (Privacidade Diferencial de Rényi)
ReLU	<i>Rectified Linear Unit</i>
SGD	<i>Stochastic Gradient Descent</i> (Gradiente Descendente Estocástico)
TV	Variação Total
UFOP	Universidade Federal de Ouro Preto
ViT	<i>Vision Transformer</i>

Sumário

2	Revisão Bibliográfica	1
2.1	Fundamentação Teórica	1
2.1.1	Aprendizado de Máquina Supervisionado	1
2.1.2	Treinamento de Redes Neurais por Gradiente	2
2.1.2.1	O Gradiente Carrega Informação sobre os Dados	3
2.1.3	Arquiteturas de Redes Neurais	5
2.1.3.1	Perceptron Multicamadas	5
2.1.3.2	Redes Neurais Convolucionais	5
2.1.3.3	Vision Transformer	6
2.1.4	Conjuntos de Dados de Referência	6
2.1.5	Aprendizado Federado	7
2.1.6	Ataques de Inversão de Gradientes	9
2.1.6.1	Deep Leakage from Gradients (DLG)	9
2.1.6.2	Improved Deep Leakage from Gradients (iDLG)	10
2.1.6.3	Inverting Gradients	11
2.1.6.4	Otimizador L-BFGS	12
2.1.6.5	Fatores que Influenciam o Sucesso do Ataque	12
2.1.7	Privacidade Diferencial	12
2.1.7.1	DP-SGD	13
2.1.7.2	Variantes de Privacidade Diferencial	15
2.1.8	Compressão e Esparsificação de Gradientes	15
2.1.8.1	Esparsificação Top- k	15
2.1.8.2	Esparsificação Random- k	16
2.1.8.3	QSGD	17
2.1.8.4	Comparação no Exemplo Guia	18
2.1.8.5	Outras Técnicas de Compressão	18
2.1.9	Métricas de Avaliação	19
2.1.9.1	Erro Quadrático Médio de Reconstrução	19
2.1.9.2	Acurácia de Inferência de Rótulo	19
2.1.9.3	Acurácia de Teste	20
2.1.9.4	Teste t de Welch e Tamanho de Efeito	20
2.2	Trabalhos Relacionados	21
2.2.1	Vazamento de Privacidade em Aprendizado Colaborativo	21
2.2.2	Ataques de Reconstrução a partir de Gradientes	22
2.2.3	Defesas e Avaliações Empíricas	23
2.2.4	Síntese e Posicionamento deste Trabalho	24

Referências 26

2 Revisão Bibliográfica

Este capítulo reúne os conceitos necessários para compreender o problema investigado e apresenta o que outros autores já estudaram sobre o tema. A Seção 2.1 constrói a base teórica passo a passo: começa pelo modo como uma rede neural aprende, passa pelo aprendizado federado e pelos ataques que exploram os gradientes, e termina nas defesas e nas métricas usadas para avaliá-las. Para tornar os conceitos mais concretos, um mesmo exemplo numérico, chamado aqui de exemplo guia, acompanha o leitor ao longo de toda a seção. A Seção 2.2 descreve os trabalhos relacionados e posiciona esta pesquisa em relação a eles.

2.1 Fundamentação Teórica

2.1.1 Aprendizado de Máquina Supervisionado

O termo aprendizado, no contexto do aprendizado de máquina, refere-se à melhoria de desempenho obtida por meio da experiência. Da mesma forma que um estudante melhora suas notas à medida que resolve mais exercícios, um programa de computador pode melhorar suas previsões à medida que analisa mais dados. [Mitchell \(1997\)](#) formaliza essa ideia: um programa aprende a partir de uma experiência E , com respeito a uma tarefa T e a uma medida de desempenho P , se seu desempenho em T , medido por P , melhora com E . Neste trabalho, a tarefa é classificar imagens, a experiência é um conjunto de imagens já rotuladas e o desempenho é a proporção de acertos em imagens que o modelo nunca viu.

Quando cada exemplo vem acompanhado da resposta correta, como uma foto acompanhada da etiqueta “gato”, o problema é chamado de aprendizado supervisionado. Formalmente, dispõe-se de um conjunto de dados $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, em que x_i é a entrada (os pixels de uma imagem) e $y_i \in \{1, \dots, K\}$ é a classe correspondente. Treinar o modelo significa encontrar os parâmetros θ de uma função f_θ que errem o mínimo possível nesses exemplos, ou seja, que minimizem o risco empírico ([Goodfellow; Bengio; Courville, 2016](#)):

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta) = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(x_i), y_i) \quad (2.1)$$

onde:

θ = vetor de parâmetros (pesos) do modelo;

$f_\theta(x_i)$ = saída do modelo para a entrada x_i ;

ℓ = função de perda, que mede o quanto a previsão se afasta da resposta correta;

N = número de exemplos de treinamento.

Em problemas de classificação com K classes, a rede produz um vetor de pontuações $z \in \mathbb{R}^K$, chamadas logits, uma para cada classe. A função softmax transforma essas pontuações em probabilidades, e a perda mais utilizada é a entropia cruzada, que penaliza o modelo quando ele atribui baixa probabilidade à classe correta:

$$p_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}, \quad \ell(z, c) = -\log p_c \quad (2.2)$$

onde:

p_j = probabilidade atribuída pelo modelo à classe j ;

c = índice da classe correta.

Se o modelo atribui probabilidade 0,9 à classe correta, a perda é $-\log 0,9 \approx 0,11$; se atribui apenas 0,1, a perda sobe para $-\log 0,1 \approx 2,30$. Essa estrutura da entropia cruzada será explorada mais adiante pelos ataques de inferência de rótulo (Seção 2.1.6.2).

2.1.2 Treinamento de Redes Neurais por Gradiente

Para redes neurais, a Equação 2.1 não tem solução direta, e os parâmetros precisam ser ajustados aos poucos. A ideia pode ser comparada a descer uma montanha no escuro: sem enxergar o vale, a pessoa sente a inclinação do terreno sob os pés e dá um passo na direção em que ele desce. Em aprendizado de máquina, essa inclinação é o gradiente $\nabla_{\theta} \mathcal{L}$, que indica a direção em que a perda mais cresce; basta, portanto, caminhar no sentido oposto.

Como calcular o gradiente sobre todos os exemplos a cada passo seria muito custoso, utiliza-se o gradiente descendente estocástico (SGD), que estima o gradiente a partir de um pequeno lote (batch) de exemplos sorteados:

$$\theta_{t+1} = \theta_t - \eta g_t, \quad g_t = \frac{1}{B} \sum_{i \in \mathcal{B}_t} \nabla_{\theta} \ell(f_{\theta_t}(x_i), y_i) \quad (2.3)$$

onde:

η = taxa de aprendizado, que controla o tamanho do passo;

$\mathcal{B}_t$ = lote de exemplos sorteado na iteração t ;

B = tamanho do lote;

g_t = gradiente médio calculado sobre o lote.

Como ilustração, considere a função $\mathcal{L}(\theta) = \theta^2$, cujo mínimo está em $\theta = 0$, com gradiente 2θ , ponto inicial $\theta_0 = 3$ e taxa $\eta = 0,1$. O primeiro passo resulta em $\theta_1 = 3 - 0,1 \times 6 = 2,4$; o segundo, em $\theta_2 = 2,4 - 0,1 \times 4,8 = 1,92$. A cada iteração, o parâmetro se aproxima do mínimo. Em uma rede neural, o mesmo processo ocorre simultaneamente para milhões de parâmetros, e o gradiente de cada um é obtido de forma eficiente pelo algoritmo de retropropagação (backpropagation), que aplica a regra da cadeia da saída para a entrada da rede (Rumelhart; Hinton; Williams, 1986).

Na prática, é comum usar variações do SGD. Uma das mais populares é o otimizador Adam (Kingma; Ba, 2015), que ajusta automaticamente o tamanho do passo de cada parâmetro com base em médias móveis do gradiente e de seu quadrado:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, & \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, & \hat{v}_t &= \frac{v_t}{1 - \beta_2^t}, \\ \theta_{t+1} &= \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \end{aligned} \quad (2.4)$$

onde:

m_t, v_t = médias móveis do gradiente e do gradiente ao quadrado;

$\hat{m}_t, \hat{v}_t$ = versões dessas médias corrigidas para o início do treinamento;

β_1, β_2 = taxas de decaimento (tipicamente 0,9 e 0,999);

ϵ = constante pequena que evita divisão por zero.

A vantagem do Adam é convergir bem sem exigir um ajuste cuidadoso da taxa de aprendizado; em contrapartida, armazena duas médias adicionais por parâmetro, o que aumenta o uso de memória.

2.1.2.1 O Gradiente Carrega Informação sobre os Dados

Um ponto central para este trabalho é que o gradiente depende diretamente dos dados usados para calculá-lo. Considere uma camada totalmente conectada $z = Wa + b$, que recebe um vetor a e produz as pontuações z . Pela regra da cadeia, o gradiente da perda em relação aos pesos e ao viés da i -ésima saída é:

$$\frac{\partial \ell}{\partial W_i} = \frac{\partial \ell}{\partial z_i} a^\top, \quad \frac{\partial \ell}{\partial b_i} = \frac{\partial \ell}{\partial z_i} \quad (2.5)$$

ou seja, cada linha do gradiente dos pesos é uma cópia da entrada a multiplicada por um número. Para ver o que isso significa na prática, considere o exemplo a seguir, que será reutilizado ao longo de todo o capítulo.

O exemplo guia consiste em um modelo com uma única camada linear que classifica entradas em três classes: avião, gato e navio. A entrada é um vetor com quatro valores não negativos, $a = (0,5; 1,0; 0,2; 2,0)$, que pode ser entendido como quatro características extraídas de uma imagem, e a classe correta é gato. A Figura 2.1 ilustra o modelo.

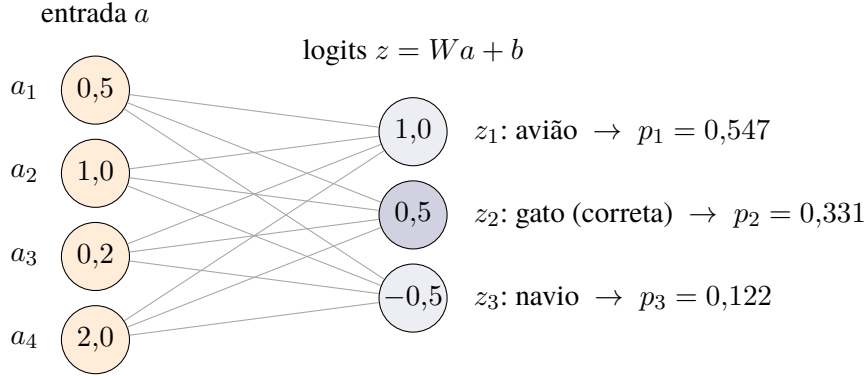


Figura 2.1 – Exemplo guia: modelo linear com quatro entradas e três classes, cuja classe correta é gato.

Suponha que os logits produzidos sejam $z = (1,0; 0,5; -0,5)$. Aplicando a softmax (Equação 2.2), obtêm-se as probabilidades $p = (0,547; 0,331; 0,122)$. O modelo erra, pois considera avião a classe mais provável, e a perda é $-\log 0,331 \approx 1,10$. Como será mostrado na Seção 2.1.6.2, a derivada da perda em relação a cada logit é p_j para as classes incorretas e $p_c - 1$ para a correta, isto é, $\partial \ell / \partial z = (0,547; -0,669; 0,122)$. Pela Equação 2.5, basta multiplicar cada um desses valores pela entrada a para obter o gradiente dos pesos:

$$\nabla W = \begin{pmatrix} 0,547 \\ -0,669 \\ 0,122 \end{pmatrix} \begin{pmatrix} 0,5 & 1,0 & 0,2 & 2,0 \end{pmatrix} = \begin{pmatrix} 0,27 & 0,55 & 0,11 & 1,09 \\ -0,33 & -0,67 & -0,13 & -1,34 \\ 0,06 & 0,12 & 0,02 & 0,24 \end{pmatrix} \begin{matrix} \leftarrow \text{avião} \\ \leftarrow \text{gato} \\ \leftarrow \text{navio} \end{matrix} \quad (2.6)$$

Agora, imagine um adversário que recebe apenas esse gradiente. Dividindo a linha de gato pelo gradiente do viés correspondente, $\partial \ell / \partial b_2 = -0,669$, ele obtém:

$$\frac{(-0,334; -0,669; -0,134; -1,337)}{-0,669} = (0,5; 1,0; 0,2; 2,0) \quad (2.7)$$

que é exatamente a entrada a . Em uma rede real, isso significa que a entrada de uma camada totalmente conectada pode ser recuperada a partir de seu gradiente (Geiping et al., 2020). O exemplo mostra, de forma simples, que o gradiente de uma camada pode conter uma cópia praticamente literal de sua entrada.

2.1.3 Arquiteturas de Redes Neurais

A forma como os neurônios de uma rede são organizados é chamada de arquitetura. Cada arquitetura parte de uma suposição diferente sobre os dados, e essa escolha afeta tanto a acurácia quanto a maneira como a informação da entrada aparece nos gradientes. Três arquiteturas são utilizadas neste trabalho.

2.1.3.1 Perceptron Multicamadas

O perceptron multicamadas (MLP) é a arquitetura mais simples: empilha várias camadas como a do exemplo guia, intercaladas por funções de ativação não lineares. Para uma entrada x transformada em vetor, a saída da camada l é:

$$h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)}), \quad h^{(0)} = x \quad (2.8)$$

onde:

$W^{(l)}, b^{(l)}$ = pesos e viés da camada l ;

σ = função de ativação, como a ReLU, $\sigma(u) = \max(0, u)$, que zera valores negativos.

A principal vantagem do MLP é a simplicidade de implementação e de análise. Sua limitação é tratar a imagem como uma lista de números, ignorando que pixels vizinhos costumam estar relacionados; por isso, precisa de muitos parâmetros para aprender padrões visuais (Goodfellow; Bengio; Courville, 2016).

2.1.3.2 Redes Neurais Convolucionais

As redes neurais convolucionais (CNN) foram criadas para aproveitar a estrutura espacial das imagens (LeCun et al., 1998). Em vez de conectar cada pixel a cada neurônio, a CNN desliza um pequeno filtro sobre a imagem, procurando o mesmo padrão (uma borda, por exemplo) em todas as posições. Essa operação é a convolução:

$$(X * K)_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X_{i+m, j+n} K_{m,n} \quad (2.9)$$

onde:

X = imagem de entrada;

K = filtro de dimensão $k \times k$, cujos valores são aprendidos no treinamento;

(i, j) = posição da saída.

Por exemplo, aplicando o filtro $\mathbf{K} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ à imagem $X = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{pmatrix}$, a primeira posição da saída é $1 \times 1 + 2 \times 0 + 0 \times 0 + 1 \times (-1) = 0$. Repetindo o cálculo nas demais posições, obtém-se $\begin{pmatrix} 0 & -1 \\ -1 & 1 \end{pmatrix}$. As camadas convolucionais costumam ser seguidas de camadas de pooling, que reduzem a resolução da imagem, e de camadas totalmente conectadas que produzem a classificação.

A grande vantagem das CNNs é reutilizar o mesmo filtro em toda a imagem, o que reduz drasticamente o número de parâmetros e as torna muito eficientes em visão computacional. Como limitação, enxergam apenas regiões locais em cada camada, precisando de várias camadas para relacionar partes distantes da imagem.

2.1.3.3 Vision Transformer

O Vision Transformer (ViT) adapta à classificação de imagens a arquitetura Transformer, criada originalmente para textos (Vaswani et al., 2017; Dosovitskiy et al., 2021). A imagem é recortada em pequenos quadrados (patches), que são tratados como se fossem as palavras de uma frase. Uma imagem do CIFAR-10, com 32×32 pixels, recortada em patches de 4×4 , gera 64 patches. Cada patch é convertido em um vetor, e o mecanismo de atenção permite que cada um consulte todos os outros para decidir quais são relevantes:

$$\text{Atenção}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (2.10)$$

onde:

Q, K, V = matrizes de consultas, chaves e valores, obtidas por projeções lineares dos patches;

d_k = dimensão das chaves, usada para normalizar as pontuações.

A vantagem do ViT é relacionar regiões distantes da imagem desde a primeira camada. Por não assumir que pixels vizinhos são parecidos, porém, precisa de mais dados para atingir boa acurácia (Dosovitskiy et al., 2021). Essa diferença estrutural também muda a forma como a informação sobre a imagem se distribui nos gradientes, o que motiva sua inclusão neste estudo.

2.1.4 Conjuntos de Dados de Referência

Para que resultados de diferentes estudos possam ser comparados, pesquisas em aprendizado de máquina costumam utilizar conjuntos de dados públicos. A Tabela 2.1 resume os três conjuntos adotados neste trabalho, em ordem crescente de dificuldade. O MNIST contém dígitos manuscritos (LeCun et al., 1998); o Fashion-MNIST mantém o mesmo formato, mas com fotos de roupas e calçados, que são mais difíceis de distinguir (Xiao; Rasul; Vollgraf, 2017); e o CIFAR-10 contém fotografias coloridas de objetos e animais, com fundos e poses variados (Krizhevsky, 2009).

Tabela 2.1 – Conjuntos de dados de imagens utilizados como referência.

Conjunto	Dimensão	Classes	Treino	Teste
MNIST	$28 \times 28 \times 1$	10	60.000	10.000
Fashion-MNIST	$28 \times 28 \times 1$	10	60.000	10.000
CIFAR-10	$32 \times 32 \times 3$	10	50.000	10.000

Fonte: [LeCun et al. \(1998\)](#), [Xiao, Rasul e Vollgraf \(2017\)](#) e [Krizhevsky \(2009\)](#).

2.1.5 Aprendizado Federado

Imagine que vários hospitais desejem treinar juntos um modelo para auxiliar diagnósticos. Reunir os prontuários de todos em um único servidor seria arriscado e, muitas vezes, proibido por lei. O aprendizado federado propõe uma alternativa: em vez de enviar os dados, cada hospital treina o modelo localmente e envia apenas as lições aprendidas, isto é, as atualizações dos parâmetros. Um servidor combina essas atualizações em um modelo comum e o devolve aos participantes ([McMahan et al., 2017](#)). Formalmente, com M clientes, cada um com n_k exemplos, o objetivo é:

$$\min_w F(w) = \sum_{k=1}^M \frac{n_k}{n} F_k(w), \quad F_k(w) = \frac{1}{n_k} \sum_{i \in \mathcal{P}_k} \ell(x_i, y_i; w) \quad (2.11)$$

onde:

w = parâmetros do modelo global;

$\mathcal{P}_k$ = conjunto de dados local do cliente k ;

n = número total de exemplos ($n = \sum_k n_k$);

F_k = perda média do cliente k sobre seus próprios dados.

O algoritmo mais conhecido é o Federated Averaging (FedAvg) ([McMahan et al., 2017](#)), que funciona em rodadas. Em cada rodada, o servidor envia o modelo atual w_t a um grupo de clientes sorteados. Cada um desses clientes treina o modelo com seus dados locais por algumas épocas, usando SGD, e envia de volta apenas os parâmetros atualizados ou os gradientes. O servidor, então, calcula a média ponderada das atualizações recebidas, dando mais peso a quem possui mais dados, e o processo se repete até que o modelo atinja a qualidade desejada.

Como exemplo da etapa de agregação, suponha três hospitais com 100, 300 e 600 pacientes, cujos treinamentos locais levaram um certo parâmetro aos valores 0,2, 0,5 e 0,1. Os pesos são 0,1, 0,3 e 0,6, e o novo valor global é $0,1 \times 0,2 + 0,3 \times 0,5 + 0,6 \times 0,1 = 0,23$. O procedimento completo é apresentado no Algoritmo 2.1 e ilustrado na Figura 2.2.

Algoritmo 2.1: Federated Averaging (FedAvg)

Entrada: fração de clientes por rodada $\mathcal{C}$, épocas locais E , tamanho do lote B , taxa de aprendizado η

Saída: parâmetros do modelo global w_T

```

1 Inicializar  $w_0$ ;
2 para cada rodada  $t = 0, 1, \dots, T - 1$  faça
3    $m \leftarrow \max(\mathcal{C} \cdot M, 1)$ ;
4    $S_t \leftarrow$  subconjunto aleatório de  $m$  clientes;
5   para cada cliente  $k \in S_t$  em paralelo faça
6      $w_{t+1}^k \leftarrow \text{ATUALIZA\_CLIENTE}(k, w_t)$ ;
7    $w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{n_{S_t}} w_{t+1}^k$ ;

8 Função  $\text{ATUALIZA\_CLIENTE}(k, w)$ 
9   para cada época local  $e = 1, \dots, E$  faça
10    para cada lote  $\mathcal{B} \subset \mathcal{P}_k$  de tamanho  $B$  faça
11       $w \leftarrow w - \eta \nabla \ell(\mathcal{B}; w)$ ;
12  retorna  $w$  ao servidor;
```

Fonte: adaptado de McMahan et al. (2017).

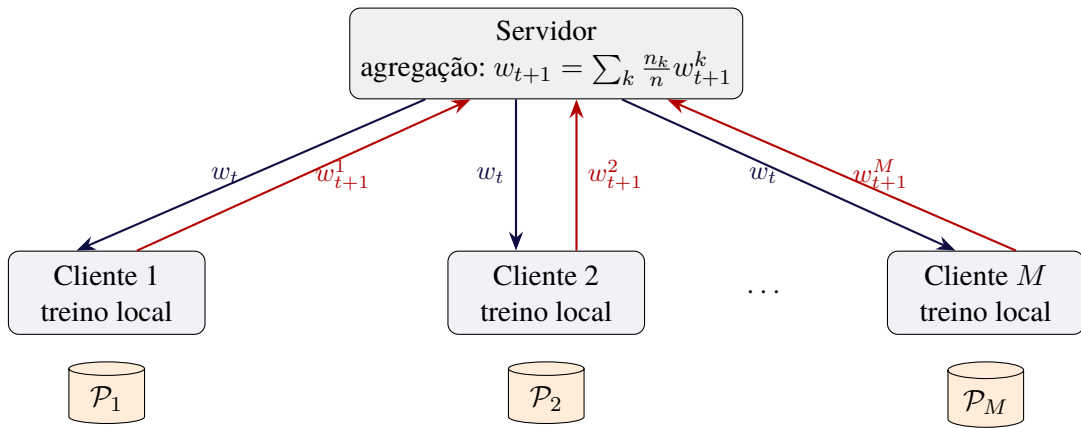


Figura 2.2 – Fluxo de uma rodada de aprendizado federado: o servidor distribui o modelo global (azul) e os clientes devolvem apenas atualizações ou gradientes (vermelho), mantendo os dados locais.

Entre as vantagens do aprendizado federado estão a redução da exposição direta dos dados e a possibilidade de treinar com informações que jamais poderiam ser centralizadas. Por outro lado, a comunicação entre clientes e servidor é custosa, e as atualizações transmitidas ainda carregam informação sobre os dados, como mostrou o exemplo guia. O modelo de ameaça mais comum considera um servidor honesto, porém curioso: ele segue o protocolo corretamente, mas tenta descobrir informações privadas a partir do que recebe (Kairouz et al., 2021). Técnicas

criptográficas como a agregação segura fazem com que o servidor conheça apenas a soma das atualizações (Bonawitz et al., 2017), mas não resolvem o problema quando poucos clientes participam de uma rodada. O pior cenário para o cliente, e por isso o mais usado nos estudos de ataque (Zhu; Liu; Han, 2019; Geiping et al., 2020), ocorre quando o adversário observa o gradiente de um único cliente calculado sobre um lote pequeno.

2.1.6 Ataques de Inversão de Gradientes

Um ataque de inversão de gradientes tenta fazer o caminho contrário ao do treinamento: em vez de usar os dados para calcular o gradiente, usa o gradiente para descobrir os dados. Uma analogia é a de um detetive que, sem ver a cena do crime, tenta reconstituí-la testando hipóteses até que as pistas produzidas por sua versão coincidam com as pistas encontradas. Supõe-se que o adversário conhece a arquitetura e os parâmetros W do modelo e observa o gradiente ∇W calculado sobre a amostra privada (x, y) . A Figura 2.3 resume a ideia.

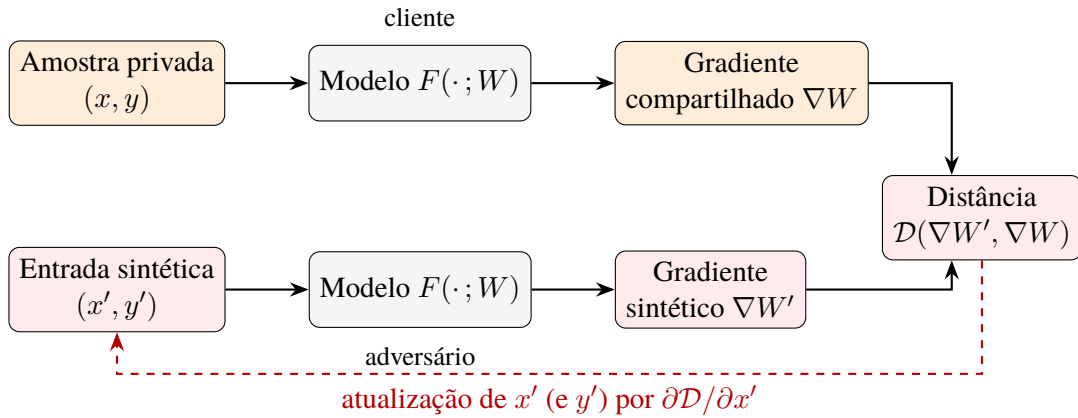


Figura 2.3 – Princípio dos ataques de inversão de gradientes baseados em otimização.

2.1.6.1 Deep Leakage from Gradients (DLG)

O DLG, proposto por Zhu, Liu e Han (2019), transforma a reconstrução em um problema de otimização. O adversário começa criando uma entrada falsa x' e um rótulo falso y' , preenchidos com valores aleatórios. Em seguida, passa (x', y') pelo modelo e calcula o gradiente $\nabla W'$ que essa entrada produziria, mede a distância entre esse gradiente e o gradiente verdadeiro ∇W e ajusta x' e y' de modo a diminuir essa distância. Esse ciclo de calcular, comparar e ajustar é repetido até que os dois gradientes praticamente coincidam, momento em que se espera que x' se pareça com a entrada original. Matematicamente, o ataque resolve:

$$x'^*, y'^* = \arg \min_{x', y'} \|\nabla W' - \nabla W\|_2^2 = \arg \min_{x', y'} \left\| \frac{\partial \ell(F(x'; W), y')}{\partial W} - \nabla W \right\|_2^2 \quad (2.12)$$

onde:

x', y' = entrada e rótulo sintéticos, ajustados pelo adversário;

∇W = gradiente compartilhado pelo cliente;

$\|\cdot\|_2^2$ = soma dos quadrados das diferenças entre os dois gradientes.

O Algoritmo 2.2 formaliza o procedimento. Os autores utilizaram o otimizador L-BFGS, descrito adiante.

Algoritmo 2.2: Deep Leakage from Gradients (DLG)

Entrada: modelo $F(\cdot; W)$, gradiente observado ∇W , número de iterações T , taxa η

Saída: amostra reconstruída (x'_T, y'_T)

```

1  $x'_0 \leftarrow \mathcal{N}(0, 1), \quad y'_0 \leftarrow \mathcal{N}(0, 1);$ 
2 para  $t = 0, \dots, T - 1$  faça
3    $\nabla W'_t \leftarrow \partial \ell(F(x'_t; W), \text{softmax}(y'_t)) / \partial W;$ 
4    $\mathcal{D}_t \leftarrow \|\nabla W'_t - \nabla W\|_2^2;$ 
5    $x'_{t+1} \leftarrow x'_t - \eta \nabla_{x'_t} \mathcal{D}_t;$ 
6    $y'_{t+1} \leftarrow y'_t - \eta \nabla_{y'_t} \mathcal{D}_t;$ 
7 retorna  $(x'_T, y'_T);$ 
```

Fonte: adaptado de [Zhu, Liu e Han \(2019\)](#).

A vantagem do DLG é a generalidade: funciona para qualquer modelo diferenciável, sem exigir conhecimento sobre os dados. Suas limitações são a instabilidade, já que otimizar entrada e rótulo ao mesmo tempo frequentemente leva a rótulos errados, e o custo computacional, pois cada iteração exige derivadas de segunda ordem da rede.

2.1.6.2 Improved Deep Leakage from Gradients (iDLG)

[Zhao, Mopuri e Bilen \(2020\)](#) perceberam que o rótulo não precisa ser adivinhado: ele pode ser lido diretamente no gradiente. Derivando a entropia cruzada (Equação 2.2) em relação às pontuações z , obtém-se:

$$\frac{\partial \ell}{\partial z_j} = p_j - \mathbb{I}[j = c] = \begin{cases} p_c - 1 \in (-1, 0), & \text{se } j = c \\ p_j \in (0, 1), & \text{se } j \neq c \end{cases} \quad (2.13)$$

onde $\mathbb{I}[j = c]$ vale 1 quando j é a classe correta e 0 caso contrário. Como toda probabilidade está entre 0 e 1, a derivada é negativa apenas para a classe correta. Pela Equação 2.5, cada linha do gradiente da última camada é essa derivada multiplicada pela entrada a ; se os valores de a forem não negativos, como ocorre após uma ReLU, o sinal da linha inteira é o sinal da derivada.

O iDLG aproveita essa propriedade da seguinte forma. Primeiro, soma os elementos de cada linha do gradiente da última camada e escolhe como rótulo a linha com a menor soma, que é a única negativa:

$$\hat{c} = \arg \min_j \sum_m (\nabla W_L)_{j,m} \quad (2.14)$$

Depois, com o rótulo fixado em $\hat{c}$, executa o DLG otimizando apenas a entrada x' , o que torna a reconstrução mais rápida e estável.

No exemplo guia, as somas das linhas do gradiente da Equação 2.6 são 2,02 para avião, $-2,47$ para gato e 0,44 para navio. A única soma negativa é a de gato, e o adversário descobre o rótulo correto sem nenhuma otimização. Note que o próprio modelo havia errado a previsão, apostando em avião; ainda assim, o gradiente revelou a classe verdadeira.

A vantagem do iDLG é tornar o ataque mais rápido e estável, com reconstruções mais fiéis que as do DLG. Sua limitação é depender de condições específicas: a regra de sinais funciona para uma única amostra e perde precisão quando o lote contém várias imagens. Por ser simples e eficaz no cenário de cliente único, o iDLG é o ataque adotado neste trabalho. Destaca-se ainda uma consequência importante: qualquer defesa que preserve o sinal das maiores entradas do gradiente tende a manter o rótulo exposto.

2.1.6.3 Inverting Gradients

Geiping et al. (2020) observaram que o tamanho do gradiente varia muito ao longo do treinamento, enquanto sua direção é o que mais revela sobre a entrada. Por isso, trocaram a distância euclidiana pela similaridade de cossenos, que compara apenas direções, e acrescentaram um termo que favorece imagens com aparência natural, a variação total (TV) (Rudin; Osher; Fatemi, 1992):

$$x'^* = \arg \min_{x' \in [0,1]^n} 1 - \frac{\langle \nabla W(x', y), \nabla W \rangle}{\|\nabla W(x', y)\| \|\nabla W\|} + \alpha \text{TV}(x') \quad (2.15)$$

$$\text{TV}(x') = \sum_{i,j} |x'_{i+1,j} - x'_{i,j}| + |x'_{i,j+1} - x'_{i,j}| \quad (2.16)$$

onde:

$\langle \cdot, \cdot \rangle$ = produto interno entre os dois gradientes;

α = peso da regularização;

$\text{TV}(x')$ = soma das diferenças entre pixels vizinhos, que é pequena em imagens suaves e grande em imagens ruidosas.

A vantagem desse método é produzir reconstruções reconhecíveis mesmo em imagens de alta resolução e em redes profundas. Em contrapartida, exige mais iterações e o ajuste do peso α .

2.1.6.4 Otimizador L-BFGS

Voltando à analogia da montanha, o SGD olha apenas para a inclinação do terreno sob os pés. O L-BFGS (Limited-memory Broyden–Fletcher–Goldfarb–Shanno) vai além: usa a memória dos últimos passos para estimar também a curvatura do terreno, o que permite escolher passos maiores quando o caminho é regular (Liu; Nocedal, 1989). Para isso, a cada iteração ele guarda as m últimas variações dos parâmetros, $s_k = \theta_{k+1} - \theta_k$, e dos gradientes, $u_k = \nabla \mathcal{L}(\theta_{k+1}) - \nabla \mathcal{L}(\theta_k)$. Com esses pares, aproxima a curvatura da função por uma matriz H_k , calcula a direção de busca $d_k = -H_k \nabla \mathcal{L}(\theta_k)$ e, por fim, escolhe o tamanho do passo por meio de uma busca linear ao longo dessa direção.

A vantagem do L-BFGS é convergir em poucas iterações em problemas suaves e de dimensão moderada, como a reconstrução de uma única imagem pequena. Sua limitação é a sensibilidade: quando a função é irregular, pode estagnar ou divergir, o que explica as falhas de convergência observadas em ataques baseados nesse otimizador.

2.1.6.5 Fatores que Influenciam o Sucesso do Ataque

A literatura aponta quatro fatores principais que tornam a inversão mais fácil ou mais difícil (Geiping et al., 2020; Huang et al., 2021). O primeiro é o tamanho do lote: quanto mais imagens contribuem para o gradiente médio, mais misturada fica a informação de cada uma. O segundo é a arquitetura, que determina quantas equações o gradiente oferece em relação ao número de pixels desconhecidos. O terceiro é o estágio do treinamento, já que modelos treinados e não treinados produzem gradientes com tamanhos muito diferentes. O quarto é o conhecimento prévio do adversário, pois saber o rótulo ou características dos dados facilita a reconstrução. As defesas descritas a seguir atuam justamente sobre o gradiente ∇W , reduzindo ou corrompendo a informação que ele carrega.

2.1.7 Privacidade Diferencial

A privacidade diferencial parte de uma pergunta simples: o resultado de uma análise mudaria muito se os dados de uma única pessoa fossem incluídos ou removidos? Se a resposta for não, quem observa o resultado não consegue descobrir se aquela pessoa participou, e sua privacidade está protegida (Dwork et al., 2006).

Uma forma intuitiva de entender a ideia é a resposta aleatorizada, usada em pesquisas sobre temas delicados. Para responder se já fez algo proibido, cada entrevistado lança uma moeda escondida: se der cara, responde a verdade; se der coroa, lança a moeda de novo e responde “sim” se der cara e “não” se der coroa. Assim, quem fez algo proibido responde “sim” com probabilidade $3/4$, e quem não fez responde “sim” com probabilidade $1/4$. Um “sim” individual nunca é prova de nada, mas, com muitos entrevistados, é possível estimar a proporção real. A razão entre essas probabilidades, 3, é justamente o que a definição formal limita.

Para enunciar essa definição, dois conjuntos de dados D e D' são chamados adjacentes quando diferem em apenas um registro. Um mecanismo aleatorizado $\mathcal{M}$ satisfaz (ε, δ) -privacidade diferencial se, para quaisquer conjuntos adjacentes D e D' e para todo subconjunto S de saídas possíveis, vale:

$$\Pr[\mathcal{M}(D) \in S] \leq e^\varepsilon \Pr[\mathcal{M}(D') \in S] + \delta \quad (2.17)$$

onde:

ε = orçamento de privacidade: quanto menor, mais parecidas são as saídas e maior a proteção;

δ = pequena probabilidade de a garantia falhar, que deve ser bem menor que $1/N$.

Na resposta aleatorizada, $e^\varepsilon = 3$, isto é, $\varepsilon = \ln 3 \approx 1,10$, com $\delta = 0$ (Dwork; Roth, 2014).

Para aplicar essa ideia a um cálculo numérico, soma-se ruído ao resultado. A quantidade de ruído depende da sensibilidade $\Delta_2 f$, que é o quanto o resultado pode mudar quando um único registro muda. No mecanismo gaussiano, o resultado $f(D)$ recebe ruído $\mathcal{N}(0, \sigma_G^2 I)$, e a garantia vale, para $\varepsilon \in (0, 1)$, quando $\sigma_G \geq \sqrt{2 \ln(1,25/\delta)} \Delta_2 f / \varepsilon$ (Dwork; Roth, 2014). Por exemplo, com sensibilidade $\Delta_2 f = 1$, $\delta = 10^{-5}$ e $\varepsilon = 0,5$, é necessário ruído com desvio-padrão de pelo menos 9,69, bem maior que a própria sensibilidade. Esse cálculo antecipa o preço da privacidade diferencial: garantias fortes exigem muito ruído.

2.1.7.1 DP-SGD

Abadi et al. (2016) adaptaram o mecanismo gaussiano ao treinamento de redes neurais, criando o DP-SGD. O problema é que o gradiente de um único exemplo pode ser arbitrariamente grande, o que tornaria a sensibilidade ilimitada. A solução combina duas operações. A primeira é o recorte (clipping): se a norma do gradiente de um exemplo ultrapassar um limite C , o gradiente é reduzido proporcionalmente até ter norma C , de modo que nenhum exemplo influencie o modelo mais do que C . A segunda é a adição de ruído: aos gradientes recortados soma-se um ruído gaussiano com desvio-padrão σC , que mascara a contribuição individual de cada exemplo.

$$\bar{g}_i = \frac{g_i}{\max\left(1, \frac{\|g_i\|_2}{C}\right)}, \quad \tilde{g} = \frac{1}{B} \left(\sum_{i=1}^B \bar{g}_i + \mathcal{N}(0, \sigma^2 C^2 I) \right) \quad (2.18)$$

onde:

g_i = gradiente do exemplo i ;

$\bar{g}_i$ = gradiente recortado;

C = limite de norma do recorte;

σ = multiplicador de ruído;

$\tilde{g}$ = gradiente privatizado, usado no lugar de g_t na Equação 2.3.

No exemplo guia, o gradiente da Equação 2.6 tem norma $\|g\|_2 = 2,01$. Com $C = 1$, cada elemento é dividido por 2,01; a linha de gato, por exemplo, passa a $(-0,17; -0,33; -0,07; -0,67)$. Em seguida, com $\sigma = 1$ e lote de uma amostra, soma-se a cada elemento um valor sorteado de uma distribuição normal com desvio-padrão 1. Em um sorteio, a linha de gato recebeu o ruído $(0,93; 0,41; 1,56; -0,89)$ e tornou-se $(0,76; 0,08; 1,49; -1,56)$. Após o mesmo procedimento nas outras linhas, as somas passaram a 1,39 para avião, 0,77 para gato e $-1,38$ para navio. Aplicando a regra do iDLG, o adversário concluiria que a classe é navio: o ruído, várias vezes maior que os valores recortados, apagou o padrão de sinais. O mesmo ruído, porém, também distorce a direção de descida usada no treinamento, e o modelo aprende pior.

O procedimento completo, que inclui a contabilização do custo de privacidade ao longo das iterações por meio do moments accountant, é apresentado no Algoritmo 2.3.

Algoritmo 2.3: DP-SGD

Entrada: dados $\{x_1, \dots, x_N\}$, perda ℓ , taxa η , multiplicador de ruído σ , lote B , limiar C , iterações T

Saída: parâmetros θ_T e custo de privacidade (ε, δ)

```

1 Inicializar  $\theta_0$  aleatoriamente;
2 para  $t = 0, \dots, T - 1$  faça
3   Sortear lote  $\mathcal{B}_t$  com probabilidade de amostragem  $B/N$ ;
4   para cada  $i \in \mathcal{B}_t$  faça
5      $g_t(x_i) \leftarrow \nabla_{\theta_t} \ell(\theta_t, x_i);$                                 // gradiente individual
6      $\bar{g}_t(x_i) \leftarrow g_t(x_i) / \max(1, \|g_t(x_i)\|_2 / C);$           // recorte
7      $\tilde{g}_t \leftarrow \frac{1}{B} (\sum_i \bar{g}_t(x_i) + \mathcal{N}(0, \sigma^2 C^2 I));$       // ruído
8      $\theta_{t+1} \leftarrow \theta_t - \eta \tilde{g}_t;$ 
9 Calcular  $(\varepsilon, \delta)$  com um contador de privacidade;
10 retorna  $\theta_T, (\varepsilon, \delta);$ 

```

Fonte: adaptado de [Abadi et al. \(2016\)](#).

A grande vantagem do DP-SGD é oferecer uma garantia matemática de privacidade, válida contra qualquer adversário, e não apenas contra um ataque específico. Sua principal limitação é o custo em utilidade: o recorte e o ruído reduzem a acurácia do modelo, especialmente quando o conjunto de dados é pequeno. Além disso, calcular gradientes individuais torna o treinamento mais lento.

2.1.7.2 Variantes de Privacidade Diferencial

A privacidade diferencial admite variações quanto a quem é protegido, a quem adiciona o ruído e a como o custo é contabilizado. A privacidade diferencial no nível do cliente, em vez de proteger um exemplo, protege todos os dados de um participante. No DP-FedAvg, por exemplo, a atualização de cada cliente é recortada e o servidor adiciona ruído à média (McMahan et al., 2018). Essa variante tem a vantagem de proteger o participante inteiro, mas exige muitos clientes por rodada para que o ruído não prejudique o modelo.

A privacidade diferencial de Rényi (RDP), por sua vez, não é uma nova defesa, mas uma forma mais precisa de somar o custo de privacidade ao longo de muitas iterações. Ela mede a diferença entre as saídas pela divergência de Rényi, exigindo $D_\alpha(\mathcal{M}(D) \parallel \mathcal{M}(D')) \leq \varepsilon$, e pode ser convertida em $(\varepsilon + \log(1/\delta)/(\alpha - 1), \delta)$ -privacidade diferencial (Mironov, 2017).

Já na privacidade diferencial local (LDP), cada cliente adiciona ruído antes de enviar seus dados, sem precisar confiar no servidor, exatamente como na resposta aleatorizada. Um mecanismo Q é ε -localmente privado se $\Pr[Q(x) \in S] \leq e^\varepsilon \Pr[Q(x') \in S]$ para quaisquer entradas x e x' . Sua vantagem é a proteção mais forte; sua limitação é o custo estatístico elevado, que reduz o tamanho efetivo da amostra de n para a ordem de $n\varepsilon^2$ (Duchi; Jordan; Wainwright, 2013).

2.1.8 Compressão e Esparsificação de Gradientes

Em treinamento distribuído, enviar gradientes com milhões de valores a cada rodada é caro. Técnicas de compressão enviam uma versão reduzida do gradiente, $\mathcal{C}(g)$, de forma parecida com a compressão de uma foto antes de enviá-la por mensagem: perde-se parte dos detalhes, mas o essencial é preservado. Embora tenham sido criadas para economizar comunicação, essas técnicas também removem parte da informação disponível ao adversário, o que motiva sua avaliação como defesa (Zhu; Liu; Han, 2019). Elas se dividem em esparsificação, que zera parte dos valores, e quantização, que reduz a precisão de cada valor.

2.1.8.1 Esparsificação Top- k

A esparsificação Top- k mantém apenas os k valores de maior magnitude do gradiente e zera os demais (Stich; Cordonnier; Jaggi, 2018). Para isso, calcula-se o valor absoluto de cada elemento, identificam-se os k elementos com os maiores valores absolutos e mantêm-se esses elementos com seus valores originais, substituindo todos os outros por zero:

$$[\mathcal{C}_{\text{top-}k}(g)]_i = \begin{cases} g_i, & \text{se } |g_i| \text{ está entre as } k \text{ maiores magnitudes de } g \\ 0, & \text{caso contrário} \end{cases} \quad (2.19)$$

A fração de valores mantidos, $\rho = k/d$, em que d é o número total de elementos, é chamada taxa de retenção. Neste trabalho, utiliza-se $\rho = 10\%$.

No exemplo guia, o gradiente da Equação 2.6 tem $d = 12$ elementos. Com $k = 3$, os maiores valores absolutos são 1,34, 1,09 e 0,67, e o gradiente comprimido mantém apenas 1,09 na linha de avião e $-0,67$ e $-1,34$ na linha de gato. As somas das linhas passam a 1,09, $-2,01$ e 0, e a regra do iDLG continua apontando gato. O motivo é que a linha da classe correta tende a concentrar os maiores valores, justamente os que o Top- k preserva; além disso, os elementos mantidos correspondem às maiores entradas ($a_4 = 2,0$ e $a_2 = 1,0$), que são as mais informativas.

A vantagem do Top- k é preservar a parte mais importante do gradiente, o que mantém a acurácia do modelo próxima à original. Stich, Cordonnier e Jaggi (2018) mostraram ainda que, guardando localmente os valores descartados e reincorporando-os nas rodadas seguintes, a convergência ocorre à mesma taxa do SGD sem compressão. Como defesa, porém, o exemplo evidencia sua limitação: ao manter os valores mais informativos, o Top- k pode manter também a informação que o adversário procura.

2.1.8.2 Esparsificação Random- k

A esparsificação Random- k também mantém k valores, mas os escolhe por sorteio, sem olhar para sua magnitude. As k posições são sorteadas com a mesma chance para todas, os valores dessas posições são mantidos e os demais são zerados. Opcionalmente, os valores mantidos podem ser multiplicados por d/k , para compensar os valores descartados:

$$[C_{\text{rand-}k}(g)]_i = \gamma \cdot g_i \cdot \mathbb{I}[i \in \Omega_k] \quad (2.20)$$

onde:

Ω_k = conjunto das k posições sorteadas;

γ = fator de escala: com $\gamma = d/k$, o gradiente comprimido é igual ao original em média; com $\gamma = 1$, os valores são enviados sem alteração.

Wangni et al. (2018) generalizaram essa ideia atribuindo a cada posição uma probabilidade própria de ser mantida, escolhida para equilibrar o ruído introduzido e o custo de comunicação.

No exemplo guia, suponha que o sorteio tenha escolhido as posições 1, 3 e 4 do gradiente da Equação 2.6, todas na linha de avião. O gradiente comprimido mantém 0,27, 0,11 e 1,09, e as somas das linhas passam a 1,47, 0 e 0. Nenhuma linha é negativa, e as linhas de gato e navio empatam: a regra do iDLG deixa de identificar o rótulo com segurança. Em outro sorteio, alguma posição da linha de gato poderia ter sido mantida, e o rótulo seria revelado; é justamente essa imprevisibilidade que dificulta o ataque.

A vantagem do Random- k como defesa é que o adversário não sabe quais partes do gradiente receberá, e a informação que chega muda a cada rodada. Sua limitação é descartar valores importantes com a mesma probabilidade que valores irrelevantes, o que torna a convergência mais lenta que a do Top- k .

2.1.8.3 QSGD

O QSGD (Quantized SGD) não zera posições; em vez disso, reduz a precisão de cada valor, arredondando-o para um de poucos níveis possíveis (Alistarh et al., 2017). É semelhante a medir alturas usando apenas as marcações de meio em meio metro. O arredondamento é feito de forma aleatória, para que, em média, o valor transmitido seja igual ao original.

O procedimento começa com o cálculo da norma $\|g\|_2$ do gradiente, que é dividida em s níveis igualmente espaçados. Para cada elemento, localizam-se os dois níveis vizinhos, l/s e $(l+1)/s$, entre os quais está sua magnitude relativa $|g_i|/\|g\|_2$. O elemento é então arredondado para cima ou para baixo por sorteio, com probabilidade de subir tanto maior quanto mais próximo ele estiver do nível superior. Por fim, transmitem-se apenas a norma, o sinal e o nível escolhido de cada elemento:

$$Q_s(g_i) = \|g\|_2 \cdot \text{sgn}(g_i) \cdot \xi_i(g, s), \quad \xi_i(g, s) = \begin{cases} (l+1)/s, & \text{com probabilidade } \frac{|g_i|}{\|g\|_2} s - l \\ l/s, & \text{caso contrário} \end{cases} \quad (2.21)$$

onde:

s = número de níveis de quantização;

l = índice do nível imediatamente abaixo de $|g_i|/\|g\|_2$;

$\text{sgn}(g_i)$ = sinal do elemento, que é sempre preservado.

No exemplo guia, com $s = 4$ e $\|g\|_2 = 2,01$, os valores possíveis são $0, \pm 0,50, \pm 1,00, \pm 1,50$ e $\pm 2,01$. Para o elemento $-1,34$ da linha de gato, a magnitude relativa multiplicada por s é $1,34/2,01 \times 4 = 2,67$; logo, $l = 2$ e a probabilidade de subir para o nível $3/4$ é $0,67$. Sorteando-se $0,58$, que é menor que $0,67$, o valor é arredondado para cima e transmitido como $-2,01 \times 3/4 = -1,50$. Já o elemento $0,27$ da linha de avião tem magnitude relativa $0,54$ e probabilidade $0,54$ de subir; sorteando-se $0,62$, é arredondado para 0 . Repetindo o processo para os doze elementos, as somas das linhas ficam $1,50, -2,50$ e $0,50$: como o sinal é preservado, o rótulo gato continua exposto, embora os valores exatos, necessários para reconstruir a entrada, tenham sido perdidos.

A vantagem do QSGD é o controle fino, pelo número de níveis s , entre economia de comunicação e perda de precisão. Como defesa, sua limitação é preservar os sinais e a ordem de grandeza dos maiores valores, o que protege o rótulo apenas parcialmente.

2.1.8.4 Comparação no Exemplo Guia

A Figura 2.4 reúne o efeito das quatro defesas sobre o gradiente do exemplo guia. Cada linha mostra os doze elementos do gradiente após a defesa, agrupados pelas classes avião, gato e navio, e a última coluna indica o rótulo que o iDLG inferiria.

	avião				gato				navio				rótulo inferido
g	0,27	0,55	0,11	1,09	-0,33	-0,67	-0,13	-1,34	0,06	0,12	0,02	0,24	gato (correto)
DP-SGD	0,18	0,73	-0,41	0,89	0,76	0,08	1,49	-1,56	0,10	-0,65	-0,77	-0,06	navio (errado)
Top-3	0	0	0	1,09	0	-0,67	0	-1,34	0	0	0	0	gato (correto)
Random-3	0,27	0	0,11	1,09	0	0	0	0	0	0	0	0	empate
QSGD ($s = 4$)	0	0,50	0	1,00	-0,50	-0,50	0	-1,50	0	0	0	0,50	gato (correto)

Figura 2.4 – Efeito das defesas sobre o gradiente do exemplo guia e rótulo inferido pela regra do iDLG. A primeira linha é o gradiente original; células em vermelho contêm ruído, células em azul são valores mantidos e zeros em cinza são valores descartados.

O exemplo resume o comportamento esperado de cada família: o ruído do DP-SGD altera todos os valores e pode inverter o padrão de sinais; o Top- k preserva justamente a região mais informativa; o Random- k pode ou não preservá-la, dependendo do sorteio; e o QSGD perde precisão, mas mantém os sinais. É importante lembrar que se trata de uma ilustração com um único sorteio e um modelo mínimo; o comportamento em redes reais precisa ser medido experimentalmente, com muitas repetições, o que é feito neste trabalho.

2.1.8.5 Outras Técnicas de Compressão

Outras técnicas relevantes, embora não avaliadas experimentalmente neste trabalho, completam o panorama. O SignSGD transmite apenas o sinal de cada elemento, $\mathcal{C}(g) = \text{sgn}(g)$, usando um único bit por valor; é extremamente econômico, mas descarta toda a informação de magnitude (Bernstein et al., 2018). O TernGrad reduz cada elemento a três níveis, $\mathcal{C}(g) = \|g\|_\infty \cdot \text{sgn}(g) \circ b$, em que b_i vale 1 com probabilidade $|g_i|/\|g\|_\infty$ e 0 caso contrário (Wen et al., 2017). O Deep Gradient Compression (DGC) combina o Top- k com o acúmulo local dos valores descartados e outras correções que preservam a acurácia mesmo com compressões muito altas (Lin et al., 2018). Por fim, a técnica de error feedback guarda o erro cometido pela compressão, $e_{t+1} = e_t + g_t - \mathcal{C}(e_t + g_t)$, e o reincorpora na rodada seguinte, corrigindo distorções de compressores como o SignSGD (Karimireddy et al., 2019).

2.1.9 Métricas de Avaliação

Avaliar uma defesa exige responder a duas perguntas que puxam em direções opostas. A primeira é quão difícil ficou para o adversário, o que corresponde à privacidade; a segunda é quão bom continua o modelo, o que corresponde à utilidade. As métricas a seguir quantificam cada uma delas.

2.1.9.1 Erro Quadrático Médio de Reconstrução

O erro quadrático médio (MSE) compara, pixel a pixel, a imagem original x com a imagem reconstruída pelo ataque x' :

$$\text{MSE}(x, x') = \frac{1}{n} \sum_{j=1}^n (x_j - x'_j)^2 \quad (2.22)$$

onde:

x_j, x'_j = valores do pixel j na imagem original e na reconstruída, na escala de intensidade dos pixels;

n = número de pixels (multiplicado pelo número de canais de cor).

Por exemplo, para uma imagem de quatro pixels $x = (0,9; 0,1; 0,8; 0,2)$, uma boa reconstrução $x' = (0,8; 0,2; 0,8; 0,3)$ tem $\text{MSE} = (0,01+0,01+0+0,01)/4 = 0,0075$, enquanto uma reconstrução ruim $x'' = (0,3; 0,7; 0,1; 0,9)$ tem $\text{MSE} = (0,36 + 0,36 + 0,49 + 0,49)/4 = 0,425$. Portanto, MSE baixo significa vazamento, e MSE alto significa proteção. A vantagem do MSE é a simplicidade e a objetividade; sua limitação é não refletir perfeitamente a percepção humana, já que uma imagem levemente deslocada pode ter MSE alto e ainda ser reconhecível.

2.1.9.2 Acurácia de Inferência de Rótulo

A acurácia de inferência de rótulo mede em quantas das amostras atacadas o rótulo deduzido pela Equação 2.14 estava correto:

$$\text{Acc}_{\text{rótulo}} = \frac{1}{A} \sum_{a=1}^A \mathbb{I}[\hat{c}_a = c_a] \quad (2.23)$$

onde:

A = número de amostras atacadas;

$\hat{c}_a, c_a$ = rótulo inferido e rótulo verdadeiro da amostra a .

Se o ataque acerta o rótulo de 9 entre 10 amostras, a métrica vale 90%. Em um problema com dez classes, um adversário que chuta ao acaso acerta cerca de 10%; valores próximos desse patamar indicam que a defesa escondeu o rótulo. Essa métrica é relevante porque, mesmo quando a imagem não é reconstruída, o rótulo já pode ser uma informação sensível, como um diagnóstico.

2.1.9.3 Acurácia de Teste

A utilidade é medida pela acurácia no conjunto de teste, isto é, a proporção de imagens não vistas no treinamento que o modelo classifica corretamente:

$$\text{Acc}_{\text{teste}} = \frac{1}{N_{\text{teste}}} \sum_{i=1}^{N_{\text{teste}}} \mathbb{I} \left[\arg \max_j f_{\theta}(x_i)_j = y_i \right] \quad (2.24)$$

onde $\arg \max_j f_{\theta}(x_i)_j$ é a classe com maior probabilidade prevista pelo modelo. Se um modelo acerta 426 de 1.000 imagens de teste, sua acurácia é 0,426. Uma boa defesa deve aumentar o MSE e reduzir a acurácia de inferência de rótulo sem derrubar a acurácia de teste.

2.1.9.4 Teste t de Welch e Tamanho de Efeito

Como os resultados variam a cada execução, por causa dos sorteios de inicialização, lotes e ruído, não basta comparar duas médias: é preciso verificar se a diferença é maior do que a variação natural. O teste t de Welch compara as médias de dois grupos independentes sem supor que tenham a mesma variabilidade (Welch, 1947):

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}, \quad \nu \approx \frac{\left(\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2} \right)^2}{\frac{(s_1^2/n_1)^2}{n_1 - 1} + \frac{(s_2^2/n_2)^2}{n_2 - 1}} \quad (2.25)$$

onde:

$\bar{x}_g, s_g^2, n_g$ = média, variância e tamanho do grupo g ;

ν = graus de liberdade aproximados, usados para obter o valor- p .

O valor- p indica a probabilidade de observar uma diferença tão grande quanto a encontrada se, na verdade, não houvesse diferença alguma. Quando ele é menor que o nível de significância α , tipicamente 0,05, conclui-se que a diferença é estatisticamente significativa. Contudo, uma diferença pode ser significativa e, ainda assim, pequena. Para medir seu tamanho, utiliza-se o d de Cohen (Cohen, 1988):

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_p}, \quad s_p = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}} \quad (2.26)$$

em que s_p é o desvio-padrão combinado dos dois grupos. Valores de $|d|$ próximos de 0,2, 0,5 e 0,8 são interpretados, respectivamente, como efeitos pequeno, médio e grande.

Como exemplo, suponha cinco medições de MSE sem defesa, (0,009; 0,004; 0,012; 0,007; 0,015), com média 0,0094 e desvio-padrão 0,0043, e cinco medições com uma defesa, (0,090; 0,020; 0,150; 0,060; 0,015), com média 0,086 e desvio-padrão 0,049. O teste de Welch resulta em $t = -3,46$, com $\nu \approx 4,1$ graus de liberdade e valor- $p \approx 0,025$, menor que 0,05: a defesa aumentou o MSE de forma significativa. O d de Cohen vale $-2,19$, indicando um efeito grande. Com apenas cinco medições por grupo, porém, o teste tem pouca sensibilidade; por isso, este trabalho utiliza até 30 observações por configuração, correspondentes a 30 execuções de treinamento na avaliação da acurácia e a 30 imagens atacadas na avaliação do MSE.

2.2 Trabalhos Relacionados

Esta seção descreve os principais trabalhos que fundamentam e motivam a presente pesquisa, organizados em três eixos: o vazamento de privacidade em aprendizado colaborativo, os ataques de reconstrução a partir de gradientes e as defesas e suas avaliações empíricas. Para cada trabalho, apresentam-se o contexto, os métodos empregados e os resultados alcançados. Ao final, a proposta deste trabalho é comparada aos estudos descritos.

2.2.1 Vazamento de Privacidade em Aprendizado Colaborativo

[Shokri e Shmatikov \(2015\)](#) propuseram um dos primeiros sistemas de aprendizado profundo colaborativo com preservação de privacidade, o Distributed Selective SGD. Nesse sistema, participantes treinam modelos locais e compartilham apenas uma fração selecionada dos gradientes, escolhida pelas maiores magnitudes ou aleatoriamente, podendo ainda adicionar ruído diferencialmente privado. Experimentos em MNIST e SVHN mostraram que o compartilhamento de uma pequena fração dos parâmetros permitia atingir acurácia próxima à do treinamento centralizado. O trabalho é relevante por antecipar, com propósito de privacidade, as mesmas operações de seleção Top- k e aleatória avaliadas nesta pesquisa.

[McMahan et al. \(2017\)](#) formalizaram o aprendizado federado e propuseram o FedAvg (Algoritmo 2.1). Em experimentos com MNIST e com modelos de linguagem sobre textos de Shakespeare, sob partições de dados não independentes e identicamente distribuídas, o FedAvg reduziu o número de rodadas de comunicação em uma a duas ordens de grandeza em relação ao FedSGD. Os autores apontaram a privacidade como motivação central, argumentando que as atualizações contêm menos informação que os dados brutos, mas sem quantificar o vazamento remanescente.

[Melis et al. \(2019\)](#) demonstraram que essa informação remanescente é explorável. Analisando as atualizações trocadas em aprendizado colaborativo, construíram ataques de inferência de pertinência (membership inference) e de propriedades, capazes de detectar se determinado

exemplo foi usado no treinamento e de inferir atributos não relacionados à tarefa principal, como características de pessoas em fotografias. Os resultados evidenciaram que os gradientes revelam mais do que o necessário para a tarefa, fenômeno denominado vazamento não intencional de atributos.

Kairouz et al. (2021) consolidaram, em ampla revisão, os avanços e problemas em aberto do aprendizado federado. O trabalho sistematiza modelos de ameaça, técnicas criptográficas como a agregação segura (Bonawitz et al., 2017), privacidade diferencial nos níveis local e central e compressão de comunicação, destacando a necessidade de avaliar conjuntamente privacidade, eficiência e utilidade.

2.2.2 Ataques de Reconstrução a partir de Gradientes

Zhu, Liu e Han (2019) apresentaram o DLG, primeiro ataque a reconstruir imagens e textos com fidelidade de pixel e de token a partir de gradientes. O método otimiza entradas e rótulos sintéticos para coincidir com o gradiente observado (Equação 2.12), usando L-BFGS. Em experimentos com uma LeNet modificada sobre MNIST, CIFAR-100, SVHN e LFW, o ataque recuperou imagens praticamente idênticas às originais em poucas centenas de iterações, inclusive para lotes de até oito imagens. Os autores avaliaram também defesas simples: ruído gaussiano ou laplaciano com variância a partir de 10^{-2} impediu a reconstrução, mas com degradação acentuada da acurácia; a representação em meia precisão não ofereceu proteção; e a poda de gradientes de pequena magnitude tornou as reconstruções irreconhecíveis a partir de cerca de 20% de esparsidade. Essas observações preliminares, obtidas em uma única arquitetura, motivam a avaliação sistemática realizada neste trabalho.

Zhao, Mopuri e Bilen (2020) propuseram o iDLG ao identificarem que o DLG frequentemente converge para rótulos incorretos. A partir da análise do sinal do gradiente da última camada sob entropia cruzada (Equações 2.13 e 2.14), demonstraram que o rótulo de uma única amostra pode ser extraído analiticamente com 100% de acerto. Em MNIST, CIFAR-100 e LFW, a extração prévia do rótulo aumentou a taxa de sucesso da reconstrução e acelerou a convergência em relação ao DLG.

Geiping et al. (2020) investigaram o quão realista é a ameaça em cenários próximos aos de implantação. Propuseram a função de custo baseada em similaridade de cossenos com regularização TV (Equação 2.15) e demonstraram analiticamente que a entrada de uma camada totalmente conectada pode ser recuperada exatamente de seu gradiente. Os experimentos, com redes ResNet treinadas e não treinadas sobre CIFAR-10 e ImageNet, mostraram reconstruções reconhecíveis de imagens de alta resolução, recuperação parcial de imagens em lotes maiores e vazamento mesmo quando os clientes executam múltiplos passos locais, como no FedAvg. Os autores concluíram que a agregação de gradientes, por si só, não garante privacidade.

Yin et al. (2021) ampliaram a escala dos ataques com o GradInversion, que recupera

lotes de imagens do ImageNet a partir de gradientes de uma ResNet-50. O método combina a restauração de rótulos do lote a partir da camada totalmente conectada, uma regularização de fidelidade baseada nas estatísticas das camadas de normalização em lote e um termo de consistência entre múltiplas reconstruções. Foram obtidas reconstruções detalhadas para lotes de até 48 imagens, evidenciando que lotes grandes não constituem, isoladamente, uma defesa suficiente.

2.2.3 Defesas e Avaliações Empíricas

Abadi et al. (2016) propuseram o DP-SGD (Algoritmo 2.3) e o moments accountant, viabilizando o treinamento de redes profundas com garantias formais de privacidade diferencial sob orçamentos moderados. Com $\delta = 10^{-5}$, os autores relataram acurácias de aproximadamente 90%, 95% e 97% no MNIST para ε igual a 0,5, 2 e 8, e de 73% no CIFAR-10 para $\varepsilon = 8$, abaixo dos valores obtidos sem privacidade. O trabalho quantificou o custo de utilidade da privacidade diferencial, mas não avaliou a resistência empírica a ataques de inversão, que foram propostos posteriormente.

McMahan et al. (2018) estenderam a privacidade diferencial ao aprendizado federado no nível do usuário com o DP-FedAvg, aplicando-o a modelos recorrentes de linguagem. Mostraram que, com um número suficientemente grande de usuários por rodada, é possível obter garantias formais com perda de acurácia pequena, à custa de maior computação. O resultado indica que o custo da privacidade diferencial depende fortemente da escala da federação, aspecto ausente em cenários com poucos clientes.

Wei et al. (2020) propuseram um arcabouço para avaliar o vazamento de dados de clientes em aprendizado federado frente a ataques de reconstrução por gradientes. Analisaram sistematicamente o efeito da inicialização do ataque, do tamanho do lote, da frequência de comunicação e do estágio do treinamento, e avaliaram defesas baseadas em adição de ruído e em compressão de gradientes em múltiplas bases de dados. Os resultados indicaram que a eficácia do ataque é sensível a esses fatores e que as defesas oferecem proteção variável conforme sua configuração, reforçando a necessidade de protocolos padronizados de avaliação.

Sun et al. (2021) propuseram o Soteria, defesa que perturba a representação dos dados em uma camada totalmente conectada escolhida, em vez de perturbar o gradiente de toda a rede. A perturbação é calculada para maximizar a distância entre a entrada original e a reconstruída, com restrição sobre o impacto na representação, e é acompanhada de análise teórica de convergência para o FedAvg. Em experimentos com MNIST e CIFAR-10 contra DLG e Inverting Gradients, o Soteria degradou a qualidade das reconstruções mantendo acurácia próxima à do modelo sem defesa.

Huang et al. (2021) avaliaram criticamente ataques e defesas de inversão de gradientes em CIFAR-10 com ResNet-18. Mostraram que parte do sucesso reportado pelos ataques decorre

de hipóteses fortes, como o conhecimento das estatísticas de normalização em lote e dos rótulos, e avaliaram defesas como poda de gradientes, MixUp e InstaHide, isoladamente e combinadas. Concluíram que a poda com taxas elevadas, especialmente combinada a outras técnicas, pode reduzir substancialmente o vazamento com perda moderada de acurácia, e recomendaram que as defesas sejam avaliadas sempre em conjunto com seu custo de utilidade.

As técnicas de compressão Top- k com memória (Stich; Cordonnier; Jaggi, 2018), Random- k (Wangni et al., 2018), QSGD (Alistarh et al., 2017), SignSGD (Bernstein et al., 2018), TernGrad (Wen et al., 2017), DGC (Lin et al., 2018) e error feedback (Karimireddy et al., 2019) foram propostas e avaliadas com foco em eficiência de comunicação. Lin et al. (2018), por exemplo, observaram que a maior parte da troca de gradientes em treinamento distribuído é redundante e obtiveram taxas de compressão de 270 a 600 vezes sem perda de acurácia em classificação de imagens, reconhecimento de fala e modelagem de linguagem. Esses trabalhos, contudo, não investigam o efeito da compressão sobre ataques de inversão, deixando em aberto se a redução de informação voltada à eficiência também se traduz em proteção.

2.2.4 Síntese e Posicionamento deste Trabalho

A análise da literatura permite identificar três lacunas. Primeiro, a maior parte dos trabalhos de ataque (Zhu; Liu; Han, 2019; Zhao; Mopuri; Bilen, 2020; Geiping et al., 2020; Yin et al., 2021) demonstra a vulnerabilidade, mas avalia defesas de forma incidental, com uma única arquitetura e sem medir sistematicamente o custo de utilidade. Segundo, os trabalhos de defesa baseados em privacidade diferencial (Abadi et al., 2016; McMahan et al., 2018) quantificam a utilidade sob garantias formais, mas não a resistência empírica a ataques de inversão; já as técnicas de compressão (Stich; Cordonnier; Jaggi, 2018; Wangni et al., 2018; Alistarh et al., 2017; Lin et al., 2018) foram concebidas para eficiência de comunicação, sem avaliação de privacidade. Terceiro, as avaliações comparativas existentes (Wei et al., 2020; Huang et al., 2021) concentram-se em arquiteturas convolucionais e em conjuntos específicos de defesas, não confrontando, sob um mesmo protocolo, privacidade diferencial e as diferentes famílias de compressão, nem arquiteturas baseadas em atenção como o ViT.

Assim como Zhu, Liu e Han (2019) e Huang et al. (2021), este trabalho avalia defesas contra ataques de inversão e considera o custo de utilidade; assim como Zhao, Mopuri e Bilen (2020), adota o iDLG, que representa um adversário capaz de inferir o rótulo analiticamente. O primeiro diferencial da proposta é comparar, sob um protocolo fatorial único, cinco configurações que abrangem as duas principais famílias de defesa: ausência de defesa, DP-SGD, Top- k , Random- k e QSGD. Além disso, o estudo inclui três arquiteturas com vieses indutivos distintos, MLP, CNN e ViT, e três bases de dados de complexidade crescente. A privacidade é medida de forma conjunta com a utilidade, a primeira pelo MSE de reconstrução e pela acurácia de inferência de rótulo e a segunda pela acurácia de teste. Por fim, a utilidade de cada configuração é medida em 30 execuções com sementes distintas, a privacidade é medida sobre 30 imagens atacadas, e as

comparações são sustentadas por testes estatísticos e tamanhos de efeito. Em contrapartida, o estudo adota um cenário de cliente único, sem agregação FedAvg, que corresponde ao caso mais favorável ao adversário discutido na Seção 2.1.5, e aplica o DP-SGD como mecanismo empírico de ruído, sem contabilização formal do orçamento (ϵ, δ) . Técnicas como privacidade diferencial no nível do cliente, LDP, SignSGD, TernGrad, DGC e error feedback foram apresentadas na fundamentação teórica, mas permanecem fora do escopo experimental.

Referências

ABADI, M.; CHU, A.; GOODFELLOW, I.; MCMAHAN, H. B.; MIRONOV, I.; TALWAR, K.; ZHANG, L. Deep learning with differential privacy. In: *Proceedings of the ACM Conference on Computer and Communications Security*. Vienna: [s.n.], 2016. p. 308–318. Citado 4 vezes nas páginas 13, 14, 23 e 24.

ALISTARH, D.; GRUBIC, D.; LI, J.; TOMIOKA, R.; VOJNOVIC, M. QSGD: Communication-efficient SGD via gradient quantization and encoding. In: *Proceedings of the Conference on Neural Information Processing Systems*. Long Beach: [s.n.], 2017. p. 1709–1720. Citado 2 vezes nas páginas 17 e 24.

BERNSTEIN, J.; WANG, Y.-X.; AZIZZADENESHELI, K.; ANANDKUMAR, A. signSGD: Compressed optimisation for non-convex problems. In: *Proceedings of the International Conference on Machine Learning*. Stockholm: [s.n.], 2018. p. 560–569. Citado 2 vezes nas páginas 18 e 24.

BONAWITZ, K.; IVANOV, V.; KREUTER, B.; MARCEDONE, A.; MCMAHAN, H. B.; PATEL, S.; RAMAGE, D.; SEGAL, A.; SETH, K. Practical secure aggregation for privacy-preserving machine learning. In: *Proceedings of the ACM Conference on Computer and Communications Security*. Dallas: [s.n.], 2017. p. 1175–1191. Citado 2 vezes nas páginas 9 e 22.

COHEN, J. *Statistical Power Analysis for the Behavioral Sciences*. 2. ed. Hillsdale: Lawrence Erlbaum Associates, 1988. Citado na página 20.

DOSOVITSKIY, A.; BEYER, L.; KOLESNIKOV, A.; WEISSENBORN, D.; ZHAI, X.; UNTERTHINER, T.; DEGHANI, M.; MINDERER, M.; HEIGOLD, G.; GELLY, S.; USZKOREIT, J.; HOULSBY, N. An image is worth 16x16 words: Transformers for image recognition at scale. In: *Proceedings of the International Conference on Learning Representations*. Virtual Event: [s.n.], 2021. Citado na página 6.

DUCHI, J. C.; JORDAN, M. I.; WAINWRIGHT, M. J. Local privacy and statistical minimax rates. In: *Proceedings of the IEEE Symposium on Foundations of Computer Science*. Berkeley: [s.n.], 2013. p. 429–438. Citado na página 15.

DWORK, C.; MCSHERRY, F.; NISSIM, K.; SMITH, A. Calibrating noise to sensitivity in private data analysis. In: *Proceedings of the Theory of Cryptography Conference*. Berlin: Springer, 2006. (Lecture Notes in Computer Science, v. 3876), p. 265–284. Citado na página 12.

DWORK, C.; ROTH, A. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, v. 9, n. 3–4, p. 211–407, 2014. Citado na página 13.

GEIPING, J.; BAUERMEISTER, H.; DRÖGE, H.; MOELLER, M. Inverting gradients – how easy is it to break privacy in federated learning? In: *Proceedings of the Conference on Neural Information Processing Systems*. Virtual Event: [s.n.], 2020. p. 16937–16947. Citado 6 vezes nas páginas 4, 9, 11, 12, 22 e 24.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. Cambridge: MIT Press, 2016. Citado 2 vezes nas páginas 1 e 5.

- HUANG, Y.; GUPTA, S.; SONG, Z.; LI, K.; ARORA, S. Evaluating gradient inversion attacks and defenses in federated learning. In: *Proceedings of the Conference on Neural Information Processing Systems*. Virtual Event: [s.n.], 2021. p. 7232–7241. Citado 3 vezes nas páginas 12, 23 e 24.
- KAIROUZ, P.; MCMAHAN, H. B.; AVENT, B.; BELLET, A.; BENNIS, M.; BHAGOJI, A. N.; BONAWITZ, K.; CHARLES, Z.; CORMODE, G.; CUMMINGS, R. et al. Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, v. 14, n. 1–2, p. 1–210, 2021. Citado 2 vezes nas páginas 8 e 22.
- KARIMIREDDY, S. P.; REBJOCK, Q.; STICH, S. U.; JAGGI, M. Error feedback fixes SignSGD and other gradient compression schemes. In: *Proceedings of the International Conference on Machine Learning*. Long Beach: [s.n.], 2019. p. 3252–3261. Citado 2 vezes nas páginas 18 e 24.
- KINGMA, D. P.; BA, J. Adam: A method for stochastic optimization. In: *Proceedings of the International Conference on Learning Representations*. San Diego: [s.n.], 2015. Citado na página 3.
- KRIZHEVSKY, A. *Learning Multiple Layers of Features from Tiny Images*. 2009. Relatório técnico. Toronto: University of Toronto. Citado 2 vezes nas páginas 6 e 7.
- LECUN, Y.; BOTTOU, L.; BENGIO, Y.; HAFFNER, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, v. 86, n. 11, p. 2278–2324, 1998. Citado 3 vezes nas páginas 5, 6 e 7.
- LIN, Y.; HAN, S.; MAO, H.; WANG, Y.; DALLY, W. J. Deep gradient compression: Reducing the communication bandwidth for distributed training. In: *Proceedings of the International Conference on Learning Representations*. Vancouver: [s.n.], 2018. Citado 2 vezes nas páginas 18 e 24.
- LIU, D. C.; NOCEDAL, J. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, v. 45, n. 1–3, p. 503–528, 1989. Citado na página 12.
- MCMAHAN, H. B.; MOORE, E.; RAMAGE, D.; HAMPSON, S.; AGÜERA Y ARCAS, B. Communication-efficient learning of deep networks from decentralized data. In: *Proceedings of the International Conference on Artificial Intelligence and Statistics*. Fort Lauderdale: [s.n.], 2017. p. 1273–1282. Citado 3 vezes nas páginas 7, 8 e 21.
- MCMAHAN, H. B.; RAMAGE, D.; TALWAR, K.; ZHANG, L. Learning differentially private recurrent language models. In: *Proceedings of the International Conference on Learning Representations*. Vancouver: [s.n.], 2018. Citado 3 vezes nas páginas 15, 23 e 24.
- MELIS, L.; SONG, C.; DE CRISTOFARO, E.; SHMATIKOV, V. Exploiting unintended feature leakage in collaborative learning. In: *Proceedings of the IEEE Symposium on Security and Privacy*. San Francisco: [s.n.], 2019. p. 691–706. Citado na página 21.
- MIRONOV, I. Rényi differential privacy. In: *Proceedings of the IEEE Computer Security Foundations Symposium*. Santa Barbara: [s.n.], 2017. p. 263–275. Citado na página 15.
- MITCHELL, T. M. *Machine Learning*. New York: McGraw-Hill, 1997. Citado na página 1.
- RUDIN, L. I.; OSHER, S.; FATEMI, E. Nonlinear total variation based noise removal algorithms. *Physica D: Nonlinear Phenomena*, v. 60, n. 1–4, p. 259–268, 1992. Citado na página 11.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, n. 6088, p. 533–536, 1986. Citado na página 3.

SHOKRI, R.; SHMATIKOV, V. Privacy-preserving deep learning. In: *Proceedings of the ACM Conference on Computer and Communications Security*. Denver: [s.n.], 2015. p. 1310–1321. Citado na página 21.

STICH, S. U.; CORDONNIER, J.-B.; JAGGI, M. Sparsified SGD with memory. In: *Proceedings of the Conference on Neural Information Processing Systems*. Montreal: [s.n.], 2018. p. 4447–4458. Citado 3 vezes nas páginas 15, 16 e 24.

SUN, J.; LI, A.; WANG, B.; YANG, H.; LI, H.; CHEN, Y. Soteria: Provable defense against privacy leakage in federated learning from representation perspective. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. Virtual Event: [s.n.], 2021. p. 9311–9319. Citado na página 23.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention is all you need. In: *Proceedings of the Conference on Neural Information Processing Systems*. Long Beach: [s.n.], 2017. p. 5998–6008. Citado na página 6.

WANGNI, J.; WANG, J.; LIU, J.; ZHANG, T. Gradient sparsification for communication-efficient distributed optimization. In: *Proceedings of the Conference on Neural Information Processing Systems*. Montreal: [s.n.], 2018. p. 1299–1309. Citado 2 vezes nas páginas 16 e 24.

WEI, W.; LIU, L.; LOPER, M.; CHOW, K.-H.; GURSOY, M. E.; TRUEX, S.; WU, Y. A framework for evaluating client privacy leakages in federated learning. In: *Proceedings of the European Symposium on Research in Computer Security*. Cham: Springer, 2020. (Lecture Notes in Computer Science, v. 12308), p. 545–566. Citado 2 vezes nas páginas 23 e 24.

WELCH, B. L. The generalization of “Student’s” problem when several different population variances are involved. *Biometrika*, v. 34, n. 1–2, p. 28–35, 1947. Citado na página 20.

WEN, W.; XU, C.; YAN, F.; WU, C.; WANG, Y.; CHEN, Y.; LI, H. TernGrad: Ternary gradients to reduce communication in distributed deep learning. In: *Proceedings of the Conference on Neural Information Processing Systems*. Long Beach: [s.n.], 2017. p. 1509–1519. Citado 2 vezes nas páginas 18 e 24.

XIAO, H.; RASUL, K.; VOLLGRAF, R. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. 2017. ArXiv:1708.07747. Citado 2 vezes nas páginas 6 e 7.

YIN, H.; MALLYA, A.; VAHDAT, A.; ALVAREZ, J. M.; KAUTZ, J.; MOLCHANOV, P. See through gradients: Image batch recovery via GradInversion. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. Virtual Event: [s.n.], 2021. p. 16337–16346. Citado 2 vezes nas páginas 22 e 24.

ZHAO, B.; MOPURI, K. R.; BILEN, H. *iDLG: Improved Deep Leakage from Gradients*. 2020. ArXiv:2001.02610. Citado 3 vezes nas páginas 10, 22 e 24.

ZHU, L.; LIU, Z.; HAN, S. Deep leakage from gradients. In: *Proceedings of the Conference on Neural Information Processing Systems*. Vancouver: [s.n.], 2019. p. 14774–14784. Citado 5 vezes nas páginas 9, 10, 15, 22 e 24.